Erlind Sejdiu
106357

SYCL

Portable heterogene Programmierung mit C++

Problemstellung, Anwendungsfälle und Programmiermodell

Seminararbeit
der Hochschule Karlsruhe
im Master Studiengang Informatik
im Wintersemester 2026/27

Betreuer: Prof. Dr.-Ing. Holger Vogelsang
Abgabedatum: 30.11.2026

Inhaltsverzeichnis

1 Problemstellung **1**

2 Forschungsfrage **1**

3 Stand der Technik **2**

4 Methode **3**

5 Konzepte des Programmiermodells **5**
5.1 Host, Gerät und Kernel 5
5.2 Speichermodell 6
5.3 Asynchrone Ausführung und Abhängigkeiten 7
5.4 Hierarchische Ausführung und lokaler Speicher 8

6 Anwendung auf die Matrixmultiplikation **10**
6.1 Stufe 0 bis 2 – Vom Programm zum Kernel 10
6.2 Stufe 2b – Buffer und Accessoren 10
6.3 Stufe 3 – Work-Groups festlegen 10
6.4 Stufe 4 – Kachelung im lokalen Speicher 11
6.5 Stufe 5 – Anpassung an das Gerät 12

7 Ergebnisse **13**

8 Diskussion **14**

9 Schlussfolgerung **15**

Literaturverzeichnis **17**

A Quellcode der Matrixmultiplikation **19**
A.1 Stufe 0 – Sequentielles Ausgangsprogramm 19
A.2 Stufe 1 – USM shared und `parallel_for` 20
A.3 Stufe 2 – USM device und explizites Kopieren 21
A.4 Stufe 2b – Buffer und Accessoren 22
A.5 Stufe 3 – Work-Groups mit `nd_range` 24
A.6 Stufe 4 – Kachelung im lokalen Speicher 25
A.7 Stufe 5 – Anpassung an das Gerät 27
A.8 Gemeinsame Hilfsfunktionen (`common.hpp`) 31

Abbildungsverzeichnis

1 SYCL-Codebasis für alle Backends 1
2 Grundaufbau (Stufe 1). Der Host reicht einen Kernel über eine Queue ein, das Gerät führt ihn als viele Work-Items aus 5
3 Speichermodell, aufbauend auf Stufe 1. Neu sind Host- und Gerätespeicher mit den Matrizen A, B, C. Bei USM device kopiert der Host die Daten ausdrücklich (durchgezogen), bei USM shared verschiebt die Laufzeit sie automatisch (gestrichelt). Stufe 1 verwendet USM shared 6
4 Speicherebenen in SYCL, aufbauend auf Abbildung 3. Neu sind die Ebenen auf dem Gerät. Nach unten werden sie kleiner, schneller und von weniger Work-Items geteilt 7
5 Getrennter Speicher und Abhängigkeiten (Stufe 2). Neu sind der Gerätespeicher und explizite Kopien, deren Abhängigkeiten als Pfeile in der Queue stehen. Kopie A und Kopie B hängen nicht voneinander ab und können gleichzeitig laufen 8
6 Eine nd_range mit globalem Bereich 8×8 und lokalem Bereich 4×4. Dick umrandet sind die vier Work-Groups 8
7 Hierarchische Ausführung (Stufen 3 und 4). Neu sind Work-Groups (Stufe 3) sowie Command Group, lokaler Speicher und Barriere (Stufe 4) . 9
8 Kachelung in Stufe 4. Eine Work-Group durchläuft die Kachelzeile von A und die Kachelspalte von B in n/T Schritten 11
9 Effizienz und Performance-Portabilität je Kachelgröße bei $n = 2048$ . . 14

Tabellenverzeichnis

1 SYCL im Vergleich zu anderen Programmiermodellen ((✓) = eingeschränkt) 2
2 Stufen der Portierung und die dabei eingeführten Konzepte 3
3 Konzepte von SYCL, die von ihnen adressierten Probleme und die Stufe, in der sie bei der Portierung eingeführt werden 9
4 Leistung je Stufe in GFLOPS (Median aus fünf Läufen, Stufe 0 auf einem Kern der jeweiligen Host-CPU) 13
5 Laufzeit je Stufe in ms (Median aus fünf Läufen). Ab Stufe 2 sind Kernel- und Transferzeit getrennt erfasst, außer bei Stufe 2b. Die Transferzeit von Stufe 3 und 5 entspricht der von Stufe 2 bzw. 4 14

1 Problemstellung

Rechenleistung liegt heutzutage in verschiedenen Formen vor. Neben der CPU stehen in fast jedem Rechner GPUs und andere technische Komponenten zur Verfügung, die datenparallele Aufgaben schnell bearbeiten können. Die Software, um diese Geräte jeweils anzusprechen, kann variieren, denn jeder Hersteller bringt sein eigenes Programmiermodell mit. Für die Software ergeben sich daraus drei Probleme.

1. **Herstellerbindung.** CUDA läuft nur auf NVIDIA-GPUs [1]. AMD (HIP) und Intel (Level Zero) haben eigene Schnittstellen.
2. **Mehrfache Codebasen.** Wer mehrere Hersteller unterstützen will, pflegt mehrere Implementierungen parallel (Abbildung 1a).
3. **Abstraktion gegen Kontrolle.** Offene Alternativen sind entweder sehr hardwarenah und aufwendig wie OpenCL [2] oder lassen wie OpenMP-Direktiven wenig Kontrolle über Speicher und Ausführung.

2 Forschungsfrage

SYCL ist ein offener Standard der Khronos Group, der diese Probleme lösen will. Mit einem einzigen Quellcode in Standard-C++ soll auf CPUs und GPUs verschiedener Hersteller dasselbe Programm laufen [3].

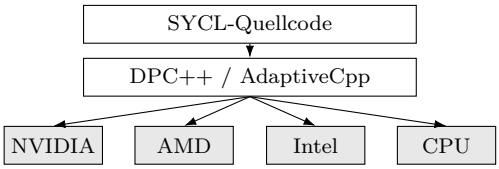


Abbildung 1: SYCL-Codebasis für alle Backends

Ob ein Programm überall läuft, ist jedoch etwas anderes, als ob es überall schnell läuft. Diese Arbeit untersucht deshalb die folgende Frage.

> Wie adressiert das Programmiermodell von SYCL die drei genannten Probleme, und erreicht ein schrittweise nach SYCL portiertes Programm neben funktionaler Portabilität auch Performance-Portabilität zwischen CPU und GPU?

Daraus ergeben sich zwei Teilfragen. (F1) Welche Konzepte des Programmiermodells werden bei einer Portierung benötigt, und welches der drei Probleme lösen sie? (F2) Welche Stufen einer Portierung lassen sich unverändert auf verschiedenen Geräten ausführen, und welche Parameter bleiben gerätespezifisch?

3 Stand der Technik

SYCL basiert auf C++17 [3]. Konkrete Implementierungen des Standards wie DPC++ [4] und AdaptiveCpp [5], [6] bilden den Standard über Backends wie CUDA, HIP, Level Zero, OpenCL oder OpenMP auf die Hardware ab und übersetzen den Gerätecode meist erst beim ersten Start just in time für das konkrete Gerät [7]. Den drei Problemen setzt SYCL vier Entwurfsziele entgegen. Portabilität (ein Quellcode, viele Backends), Offenheit (ein Standard mit mehreren unabhängigen Implementierungen), Produktivität (Host- und Gerätecode in einer Datei in Standard-C++) und Kontrolle (Hardwarekonzepte wie lokaler Speicher bleiben zugänglich). Tabelle 1 ordnet SYCL entlang dieser Ziele ein. Kokkos [8] und RAJA [9] verfolgen ähnliche Ziele, sind aber kein offener Industriestandard.

	herstellerunabh.	**offener Standard**	**Standard-C++**	**Kontrolle**
CUDA	–	–	(✓)	✓
OpenCL	✓	✓	–	✓
OpenMP-Offload	✓	✓	✓	(✓)
Kokkos/RAJA	✓	–	✓	(✓)
SYCL	✓	✓	✓	✓

Tabelle 1: SYCL im Vergleich zu anderen Programmiermodellen ((✓) = eingeschränkt)

Eingesetzt wird SYCL vor allem im wissenschaftlichen Rechnen auf Systemen verschiedener Hersteller, etwa auf dem Supercomputer Aurora oder in GROMACS, bei der Migration von CUDA-Anwendungen mit dem Werkzeug SYCLomatic und in portablen Bibliotheken wie oneMKL [10]. Frühere Studien fanden SYCL auf vielen Plattformen konkurrenzfähig mit herstellerspezifischem Code, aber deutlich abhängig von Implementierung und Parametrierung [11], [12]. Als Maß für Performance-Portabilität hat sich die Kennzahl von Pennycook, Sewall und Lee etabliert [13]. Offen bleibt in diesen Arbeiten meist, welcher Schritt einer Portierung welchen Effekt auf welchem Gerät hat. Hier setzt die vorliegende Seminararbeit an.

4 Methode

Untersucht wird die Multiplikation $C = A \cdot B$ zweier $n \times n$-Matrizen. Das Problem benötigt $2n^3$ Gleitkommaoperationen bei $3n^2$ Werten Datenvolumen, ist gut parallelisierbar und leicht auf Korrektheit prüfbar. Ein sequentielles C++-Programm wird in Stufen nach SYCL überführt, wobei jede Stufe nur so viele neue Konzepte einführt, wie für ihren Schritt nötig sind (Tabelle 2). Abschnitt 5 erklärt diese Konzepte zunächst allgemein und in der Reihenfolge, in der sie gebraucht werden. Abschnitt 6 zeigt anschließend, wie sie in den einzelnen Stufen umgesetzt sind. Das beantwortet F1. Zugleich lässt sich die Wirkung jedes Schritts getrennt messen, was F2 beantwortet.

Stufe	**Änderung**	**Neue Konzepte**
0	Sequentielles Ausgangsprogramm auf einem CPU-Kern	–
1	Ein Work-Item je Element von C	Host und Gerät, Queue, Kernel, Indexraum, USM shared, Backends, Geräteauswahl
2	Getrennter Gerätespeicher, explizites Kopieren	USM device, explizite Kopien, Events, Abhängigkeiten
2b	Variante von Stufe 2: Buffer statt Zeiger, Kopieren implizit	Buffer, Accessoren, automatische Abhängigkeiten
3	Work-Group-Größe selbst festlegen, noch ohne lokalen Speicher	nd_range, Work-Group
4	Kachelung im lokalen Speicher mit fester Kachelgröße	Command Group, lokaler Speicher, Barriere
5	Kachelgröße zur Laufzeit passend zum Gerät wählen	Geräteabfragen, Specialization Constants, synchrone und asynchrone Fehlerbehandlung

Tabelle 2: Stufen der Portierung und die dabei eingeführten Konzepte

Gemessen wird mit DPC++ [4] auf drei Geräten. Zumal auf einem AMD Ryzen 5 5600X (sechs Kerne, OpenCL-Backend), einer NVIDIA GeForce RTX 5060 Ti und einer NVIDIA L40S auf dem DACHS-Cluster der Hochschule Esslingen (beide CUDA-Backend). Jede Stufe wird einmal übersetzt und unverändert auf allen drei Geräten ausgeführt. Das Gerät wird beim Start über die Umgebungsvariable `ONEAPI_DEVICE_SELECTOR` gewählt, etwa `cuda:gpu` oder `opencl:cpu`. Stufe 4 wird zusätzlich für die Kachelgrößen $T \in \{8, 16, 32, 64\}$ übersetzt und gemessen, Stufe 5 wählt T selbst. Jede Stufe läuft für $n \in \{256, 512, 1024, 2048\}$. Nach einem getrennt erfassten ersten Lauf, der die JIT-Kompilierung enthält, folgen fünf Messläufe, von denen der Median berichtet wird.

Die Zeit misst der Host mit der monotonen Uhr der C++-Standardbibliothek (`std::chrono::steady_clock`) vom Einreichen eines Auftrags bis zum Ende des Wartens auf ihn (A.8, Z. 44–53). Ab Stufe 2 werden Kernel und Kopien so getrennt erfasst.

Gemessen wird damit nicht nur die Ausführung auf dem Gerät, sondern auch das Einreichen und der Start des Kernels, was nur bei kleinen Matrizen ins Gewicht fällt. SYCL bietet als Alternative eine Messung auf dem Gerät selbst. Eine Queue, die mit der Eigenschaft `enable_profiling` angelegt ist, liefert zu jedem Event die Zeitpunkte, zu denen der Auftrag auf dem Gerät begann und endete [3]. Die Host-Messung wurde gewählt, weil sie für alle Stufen gleich funktioniert, auch für das sequentielle Ausgangsprogramm und für Stufe 2b, deren Kopien die Laufzeit selbst einplant.

5 Konzepte des Programmiermodells

Dieses Kapitel erklärt die Konzepte von SYCL unabhängig von der Matrixmultiplikation. Die Reihenfolge folgt den Stufen aus Tabelle 2, sodass jedes Konzept dort steht, wo es bei der Portierung zum ersten Mal gebraucht wird. Die Abbildungen 2, 3, 5 und 7 bauen aufeinander auf. Bereits Bekanntes ist grau gezeichnet, Neues dick schwarz. Wie die Konzepte im Quellcode der Matrixmultiplikation umgesetzt sind, zeigt Abschnitt 6.

5.1 Host, Gerät und Kernel

SYCL unterscheidet zwei Rollen. Der Host ist die CPU, auf der das Hauptprogramm läuft. Ein Gerät ist die Hardware, die rechnet, etwa eine GPU oder auch die CPU selbst. Ein Kernel ist eine Funktion, die auf dem Gerät vielfach parallel ausgeführt wird. Jede einzelne Ausführung heißt Work-Item. Wie viele Work-Items es gibt und wie sie angeordnet sind, legt der Indexraum fest. Jedes Work-Item kennt seine Position im Indexraum und wählt darüber die Daten aus, die es bearbeitet. Ein Kernel beschreibt also nur, was ein einzelnes Work-Item tut. Der Host übergibt Kernel nicht direkt an das Gerät, sondern über eine Queue. Jede Queue ist mit genau einem Gerät verbunden (Abbildung 2). Das Einreichen ist asynchron. Der Aufruf kehrt sofort zurück, und der Host kann weiterarbeiten, bis er das Ergebnis tatsächlich braucht und ausdrücklich darauf wartet.

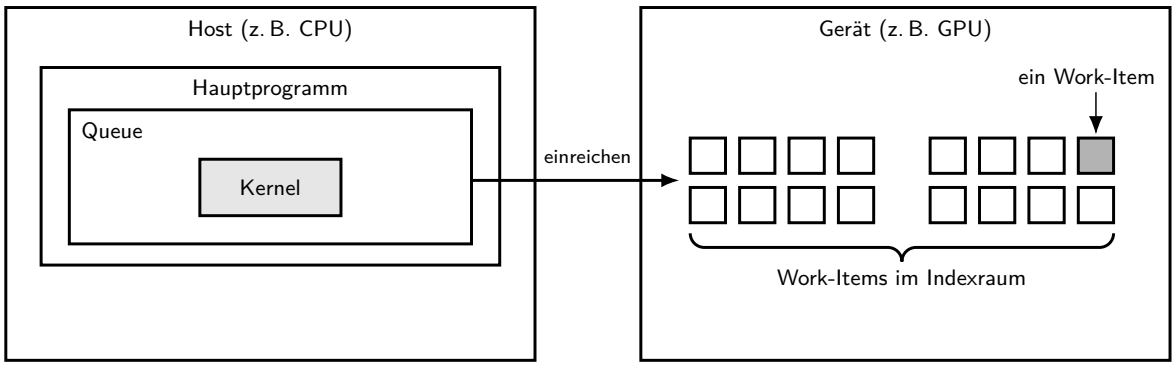


Abbildung 2: Grundaufbau (Stufe 1). Der Host reicht einen Kernel über eine Queue ein, das Gerät führt ihn als viele Work-Items aus

Host- und Gerätecode stehen in derselben C++-Quelldatei (Single-Source) [3]. Der Kernel wird meist als Lambda-Ausdruck geschrieben und kann Variablen des Hauptprogramms als Kopie übernehmen. Der Compiler erkennt den Gerätecode und übersetzt ihn zusätzlich für die Zielgeräte, entweder in einem eigenen Durchlauf je Ziel oder, wie AdaptiveCpp, in einem einzigen gemeinsamen Durchlauf [7]. Weil Geräte keine vollständige C++-Umgebung besitzen, gelten im Kernel Einschränkungen. Ausnahmen, Rekursion, virtuelle Funktionsaufrufe und dynamische Speicheranforderung sind dort

nicht erlaubt. Gegenüber CUDA oder OpenCL entfällt damit eine eigene Kernelsprache. Es bleibt bei einer Codebasis in Standard-C++.

Jedes Gerät spricht die Implementierung über ein Backend an, etwa CUDA für NVIDIA-GPUs, HIP für AMD-GPUs, Level Zero für Intel-GPUs oder OpenCL für CPUs. Welches Gerät eine Queue nutzt, bestimmt ein Device Selector, der jedem verfügbaren Gerät eine Bewertung gibt und das am besten bewertete wählt. Die ausführbare Datei enthält den Gerätecode dazu in Zwischendarstellungen wie PTX für CUDA oder SPIR-V für OpenCL, die der Treiber erst beim ersten Start eines Kernels just in time für das konkrete Gerät übersetzt [7]. So läuft dieselbe Datei ohne Neuübersetzung auf Geräten verschiedener Hersteller, der erste Lauf ist dafür langsamer als alle folgenden.

5.2 Speichermodell

Host und Gerät besitzen häufig getrennte Speicher, etwa den Arbeitsspeicher des Rechners und den Grafikspeicher einer GPU. SYCL bietet zwei Wege, Daten zwischen ihnen zu verwalten, Unified Shared Memory (USM) und Buffer.

USM arbeitet mit gewöhnlichen C++-Zeigern, die je nach Variante auf Host, Gerät oder beiden gültig sind [3]. Bei USM host liegen die Daten im Host-Speicher, und das Gerät greift über den Bus darauf zu. Bei USM shared gilt derselbe Zeiger auf Host und Gerät. Die Laufzeit verschiebt die Daten automatisch dorthin, wo zugegriffen wird. Das ist bequem, verbirgt aber, wann und wie viel kopiert wird. Bei USM device liegen die Daten nur im Gerätespeicher, und der Host kopiert sie ausdrücklich hin und zurück. Das verlangt mehr Code, macht den Transfer aber sichtbar und steuerbar (Abbildung 3).

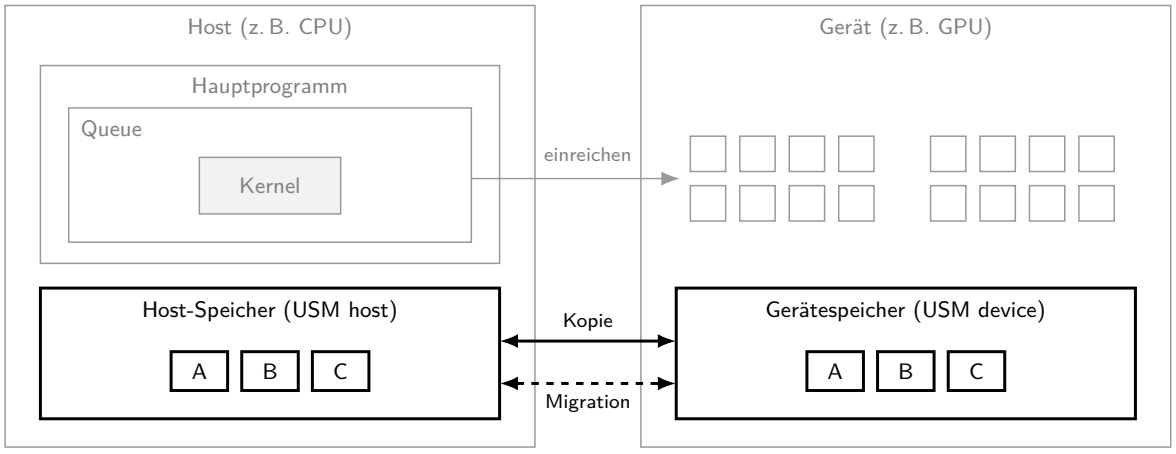


Abbildung 3: Speichermodell, aufbauend auf Stufe 1. Neu sind Host- und Gerätespeicher mit den Matrizen A, B, C. Bei USM device kopiert der Host die Daten ausdrücklich (durchgezogen), bei USM shared verschiebt die Laufzeit sie automatisch (gestrichelt). Stufe 1 verwendet USM shared

Buffer sind die ältere, abstraktere Alternative. Ein Buffer kapselt die Daten, und ein

Kernel greift nur über Accessoren darauf zu, die zugleich angeben, ob gelesen, geschrieben oder beides wird. Aus diesen Angaben leitet die Laufzeit selbst ab, wann Daten kopiert werden müssen und welche Kernel voneinander abhängen. Der Programmierer verliert dafür die direkte Kontrolle über Zeitpunkt und Umfang der Kopien [10].

Auf dem Gerät selbst gibt es mehrere Speicherebenen (Abbildung 4). Der globale Speicher ist groß, für alle Work-Items sichtbar, aber vergleichsweise langsam. Der lokale Speicher ist klein und schnell und wird nur von den Work-Items einer Work-Group geteilt (Abschnitt 5.4). Der private Speicher, auf der GPU meist Register, gehört einem einzelnen Work-Item. Je weiter unten eine Ebene liegt, desto schneller und kleiner ist sie. Daten so zu verteilen, dass häufig gelesene Werte in den schnellen Ebenen liegen, ist der wichtigste Hebel für Leistung [14].

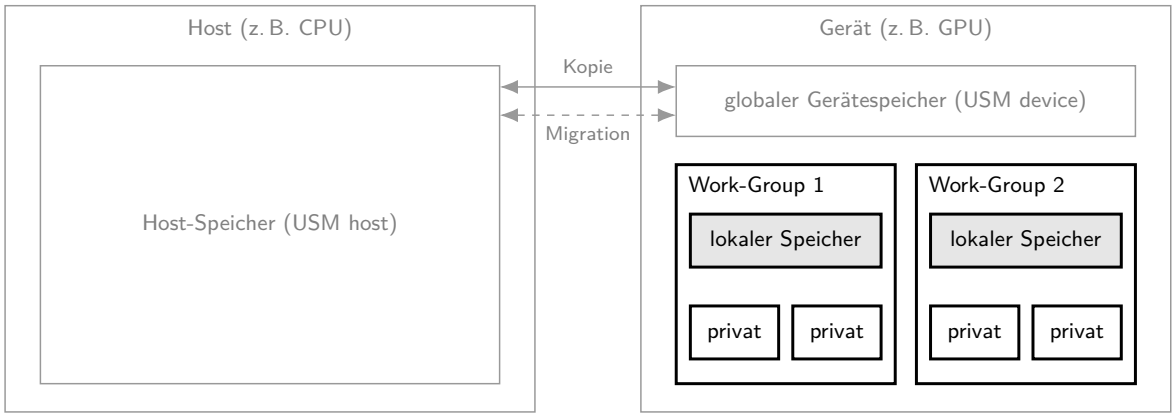


Abbildung 4: Speicherebenen in SYCL, aufbauend auf Abbildung 3. Neu sind die Ebenen auf dem Gerät. Nach unten werden sie kleiner, schneller und von weniger Work-Items geteilt

5.3 Asynchrone Ausführung und Abhängigkeiten

Da Kopien und Kernel asynchron eingereicht werden, muss die Laufzeit wissen, in welcher Reihenfolge sie ausgeführt werden dürfen. Ein Kernel darf zum Beispiel erst starten, wenn seine Eingabedaten auf dem Gerät angekommen sind. Jeder Auftrag an die Queue liefert dafür ein Event. Spätere Aufträge können angeben, auf welche Events sie warten. Aus diesen Abhängigkeiten baut die Laufzeit einen gerichteten azyklischen Graphen und führt Aufträge, die nicht voneinander abhängen, gleichzeitig aus (Abbildung 5) [10].

Eine Queue ist standardmäßig out-of-order. Sie darf Aufträge in beliebiger Reihenfolge ausführen, solange die angegebenen Abhängigkeiten eingehalten werden. Eine in-order-Queue führt dagegen alle Aufträge streng nacheinander aus. Das ist einfacher, verschenkt aber Parallelität zwischen unabhängigen Aufträgen. Wer Buffer verwendet, muss Abhängigkeiten nicht selbst angeben, weil die Laufzeit sie aus den Accessoren ableitet. Die gröbste Form der Synchronisation ist das Warten des Hosts auf die gesamte Queue. Danach ist sicher jeder bisher eingereichte Auftrag beendet.

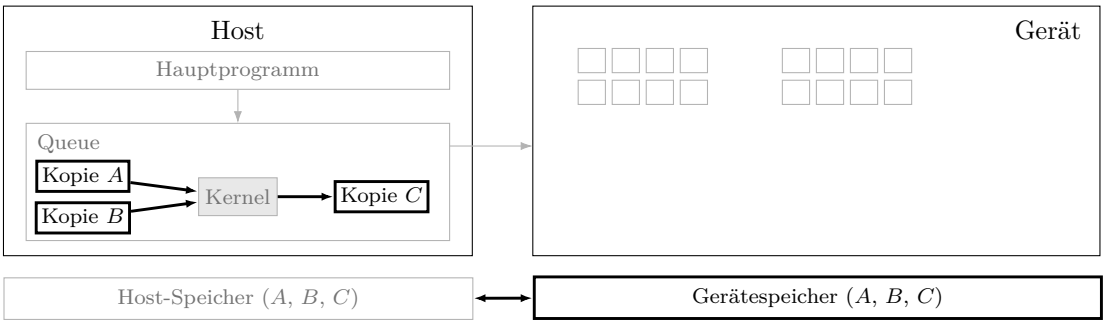


Abbildung 5: Getrennter Speicher und Abhängigkeiten (Stufe 2). Neu sind der Gerätespeicher und explizite Kopien, deren Abhängigkeiten als Pfeile in der Queue stehen. Kopie A und Kopie B hängen nicht voneinander ab und können gleichzeitig laufen

5.4 Hierarchische Ausführung und lokaler Speicher

Im einfachen Indexraum aus Abschnitt 5.1 sind alle Work-Items gleichberechtigt und können nicht zusammenarbeiten. Für Optimierungen erlaubt SYCL deshalb einen zweistufigen Indexraum, die nd_range. Sie besteht aus einem globalen Bereich und einem lokalen Bereich, der den globalen in gleich große Work-Groups aufteilt (Abbildung 6). Der globale Bereich muss dafür in jeder Dimension ein Vielfaches des lokalen sein. Jedes Work-Item kennt nun drei Positionen, seine globale ID im gesamten Indexraum, seine lokale ID innerhalb der Work-Group und die ID seiner Work-Group. Beim einfachen Indexraum teilt die Laufzeit die Work-Items ebenfalls in Work-Groups auf, wählt deren Größe aber selbst. Erst die nd_range gibt diese Entscheidung an das Programm.

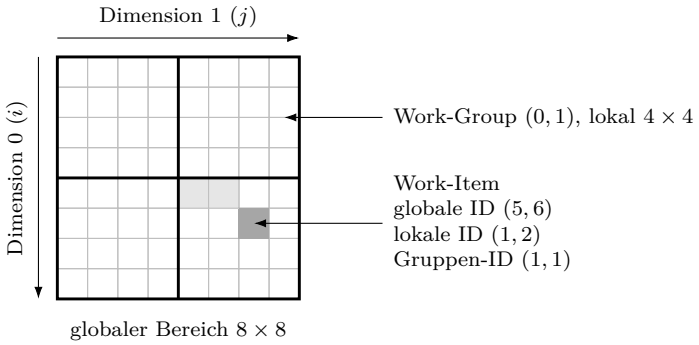


Abbildung 6: Eine nd_range mit globalem Bereich 8×8 und lokalem Bereich 4×4. Dick umrandet sind die vier Work-Groups

Die Work-Items einer Work-Group laufen auf derselben Recheneinheit und teilen sich deren lokalen Speicher (Abbildung 4). Sie können damit zusammenarbeiten. Typischerweise lädt jedes Work-Item einen Teil der Daten aus dem globalen in den lokalen Speicher, und anschließend lesen alle Work-Items die gemeinsam geladenen Werte. Damit kein Work-Item einen Wert liest, bevor ein anderes ihn geschrieben hat, synchronisieren sie sich an einer Barriere. Jedes Work-Item wartet dort, bis alle Work-Items

seiner Gruppe sie erreicht haben, und erst danach sind die Schreibzugriffe der anderen sichtbar. Eine Barriere über Work-Groups hinweg gibt es nicht. Work-Groups laufen unabhängig voneinander und in beliebiger Reihenfolge, weshalb sie nur über das Ende eines Kernels synchronisiert werden können.

Lokaler Speicher wird nicht im Kernel angelegt, sondern vor dem Start reserviert, da die Laufzeit seinen Bedarf kennen muss, bevor sie die Work-Groups auf die Recheneinheiten verteilt. Der Host reicht den Kernel dazu zusammen mit dieser Reservierung als Command Group ein (Abbildung 7). Eine Command Group bündelt einen Auftrag mit allem, was er braucht, also Speicher, Accessoren und Abhängigkeiten. Die Kurzformen aus den vorigen Abschnitten sind nur eine Abkürzung dafür.

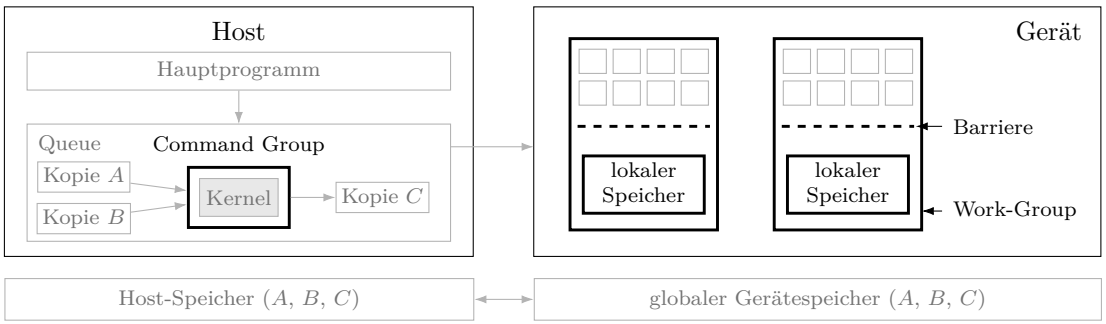


Abbildung 7: Hierarchische Ausführung (Stufen 3 und 4). Neu sind Work-Groups (Stufe 3) sowie Command Group, lokaler Speicher und Barriere (Stufe 4)

Konzept	**adressiertes Problem**	**Stufe**
Single-Source in Standard-C++	mehrfache Codebasen	1
Backends, Device Selector, JIT-Kompilierung	Herstellerbindung	1
Queue, Kernel, Indexraum, USM shared	Abstraktion (einfacher Einstieg)	1
USM device, Kopien, Events, Abhängigkeiten	Kontrolle über Datentransfer	2
Buffer und Accessoren	Abstraktion (Abhängigkeiten automatisch)	2b
nd_range, Work-Group	Kontrolle über Ausführung	3
Command Group, lokaler Speicher, Barriere	Kontrolle über Speicherebenen	4
Geräteabfragen, Specialization Constants, Fehlerbehandlung	Kontrolle ohne Verlust an Portabilität	5

Tabelle 3: Konzepte von SYCL, die von ihnen adressierten Probleme und die Stufe, in der sie bei der Portierung eingeführt werden

6 Anwendung auf die Matrixmultiplikation

Dieses Kapitel zeigt, wie die Konzepte aus Abschnitt 5 im Programm umgesetzt sind. Verweise der Form (A.2, Z. 36–43) bezeichnen Abschnitt und Zeilen im Quellcode des Anhangs A. Alle Stufen teilen sich die Hilfsfunktionen für Zeitmessung, Prüfung und Ausgabe (A.8, Z. 1–90) und unterscheiden sich nur an den hier beschriebenen Stellen.

6.1 Stufe 0 bis 2 – Vom Programm zum Kernel

Das Ausgangsprogramm berechnet C mit drei verschachtelten Schleifen über zeilenweise gespeicherte Matrizen (A.1, Z. 12–20). Die Elemente von C sind voneinander unabhängig, und genau das nutzt Stufe 1. Sie legt eine Queue mit dem Standard-Selektor an (A.2, Z. 24), die Matrizen in USM shared (A.2, Z. 28–30) und ersetzt die Schleifen über i und j durch ein `parallel_for` über einen Indexraum $n \times n$ (A.2, Z. 36–43). Jedes Work-Item berechnet ein Element von C, die Schleife über k bleibt im Kernel. Da die Zeiger auch auf dem Gerät gültig sind, ist keine Zeile für den Datentransfer nötig.

Stufe 2 lässt den Kernel unverändert (A.3, Z. 43–50) und trennt den Speicher. Die Matrizen liegen als Vektoren auf dem Host (A.3, Z. 28–30) und als USM device auf dem Gerät (A.3, Z. 33–35), und der Host kopiert ausdrücklich hin und zurück (A.3, Z. 37–41, 52). Statt über Events wartet er nach jedem Abschnitt auf die gesamte Queue (A.3, Z. 40, 50, 52). Das verschenkt Überlappung, macht Kopier- und Rechenzeit aber getrennt messbar.

6.2 Stufe 2b – Buffer und Accessoren

Die Variante 2b geht denselben Schritt mit Buffern. Sie kapseln die Host-Felder (A.4, Z. 36–38), und Accessoren geben an, dass A und B nur gelesen und C nur geschrieben wird (A.4, Z. 41–43). Daraus plant die Laufzeit Kopien und Abhängigkeiten selbst. Erst der Destruktor des Buffers von C kopiert das Ergebnis zurück (A.4, Z. 54). Der Code wird kürzer und sicherer, Kopien und Kernel sind aber nur noch gemeinsam messbar (A.4, Z. 60–66).

6.3 Stufe 3 – Work-Groups festlegen

Stufe 3 ersetzt den einfachen Indexraum durch eine nd_range mit Work-Groups aus $W \times W = 16 \times 16$ Work-Items (A.5, Z. 30, 53). Da der globale Bereich ein Vielfaches des lokalen sein muss, wird n aufgerundet (A.5, Z. 44), und überzählige Work-Items kehren sofort zurück (A.5, Z. 57). Die Rechnung bleibt die aus Stufe 2. Was sich ändert, geht damit allein auf die selbst gewählte Work-Group-Größe zurück.

6.4 Stufe 4 – Kachelung im lokalen Speicher

Bis Stufe 3 liest jedes Work-Item $2n$ Werte aus dem globalen Speicher, und benachbarte Work-Items lesen dieselben Werte immer wieder. Stufe 4 zerlegt die Matrizen deshalb in Kacheln der Größe $T \times T$. Jede Work-Group berechnet eine Kachel von C und lädt dafür in n/T Schritten je eine Kachel von A und B gemeinsam in den lokalen Speicher (Abbildung 8). Jedes Work-Item liest so nur noch $2n/T$ Werte aus dem globalen Speicher.

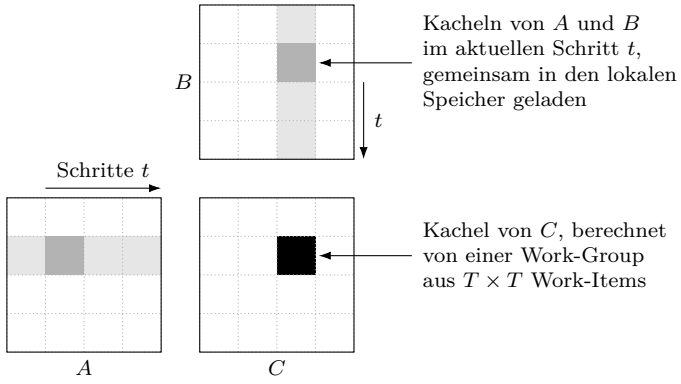


Abbildung 8: Kachelung in Stufe 4. Eine Work-Group durchläuft die Kachelzeile von A und die Kachelspalte von B in n/T Schritten

Weil lokaler Speicher vor dem Start reserviert werden muss, wird der Kernel als Command Group eingereicht, die zwei lokale Felder für die Kacheln anlegt (A.6, Z. 60–63). Die Work-Group-Größe der nd_range entspricht nun der Kachelgröße (A.6, Z. 66). In der Schleife über die Kacheln (Listing 1) lädt jedes Work-Item ein Element von A und B (Zeilen 76–77). Die erste Barriere stellt sicher, dass beide Kacheln vollständig geladen sind, bevor gerechnet wird. Die zweite verhindert, dass ein schnelles Work-Item die Kachel überschreibt, während andere noch mit ihr rechnen. Elemente außerhalb der Matrix werden als null geladen und nicht geschrieben.

Listing 1: Schleife über die Kacheln in Stufe 4 (Ausschnitt aus Anhang A.6)

```
73          float sum = 0.0f;
74          for (std::size_t t = 0; t < n; t += T) {
75            // gemeinsam eine Kachel laden (außerhalb der Matrix: 0)
76            tileA[li][lj] = (i < n && t + lj < n) ? A[i * n + t + lj] : 0.0f;
77            tileB[li][lj] = (t + li < n && j < n) ? B[(t + li) * n + j] : 0.0f;
78            sycl::group_barrier(it.get_group());
79
80            // mit den Daten aus dem schnellen Local Memory rechnen
81            for (std::size_t k = 0; k < T; ++k)
82              sum += tileA[li][k] * tileB[k][lj];
83            sycl::group_barrier(it.get_group());
84          }
85          if (i < n && j < n) C[i * n + j] = sum;
```

T wird beim Übersetzen gesetzt (A.6, Z. 28–30, 36), sodass für jedes $T \in \{8, 16, 32, 64\}$ ein eigenes Programm entsteht. Bei $T = 64$ überschreitet eine Work-Group mit 4096 Work-Items die GPU-Grenze von 1024 Threads je Block [15]. Die Laufzeit wirft beim Einreichen einen synchronen Fehler, den Stufe 4 nicht abfängt, und das Programm bricht ab. Auf der CPU läuft $T = 64$ fehlerfrei.

6.5 Stufe 5 – Anpassung an das Gerät

Stufe 5 wählt T zur Laufzeit. Der Kernel aus Stufe 4 liest T dazu aus einer Specialization Constant (A.7, Z. 30, 62), die der Host vor dem Einreichen setzt (A.7, Z. 49). Für den JIT-Compiler bleibt T eine Konstante. Für AdaptiveCpp, das den Mechanismus nicht unterstützt, enthält das Programm einen zweiten Weg über dessen Erweiterung (A.7, Z. 42–63). Ein Async Handler gibt Fehler vom Gerät als Ausnahme weiter (A.7, Z. 93–98), sodass synchrone und asynchrone Fehler im selben `try`-Block landen. Über Geräteabfragen liest das Programm die maximale Work-Group-Größe und den lokalen Speicher (A.7, Z. 103–106) und verwirft jedes T, das nicht hineinpasst (A.7, Z. 139–144). Auf den GPUs fällt so $T = 64$ weg. Die übrigen Kachelgrößen misst es nach einem Aufwärmlauf mit JIT-Übersetzung je dreimal und wählt die schnellste. Scheitert ein Kernel trotz Abfrage, etwa wegen zu vieler Register, fängt der `catch`-Block das ab, und das Programm versucht die nächste (A.7, Z. 145–161). Die Auswahl zählt zum ersten Lauf, nicht zu den Messläufen.

7 Ergebnisse

Alle Stufen liefen ohne Codeänderung und mit korrektem Ergebnis auf allen drei Geräten. [für 2b, 3 und 5 nach der Messung prüfen] Tabelle 4 zeigt die Wirkung jeder Stufe. Schon Stufe 1 ist bei $n = 1024$ rund 2300-mal (RTX) bzw. 10 700-mal (L40S) schneller als das sequentielle Ausgangsprogramm auf einem CPU-Kern. Dieser Vergleich misst allerdings Parallelisierung und Gerätewechsel zugleich. Stufe 2 ändert die Rechenzeit nicht, macht aber den Transfer sichtbar, der bei $n = 2048$ ein Viertel (RTX) bis ein Drittel (L40S) der Rechenzeit ausmacht. Stufe 2b braucht für Kopien und Kernel zusammen [Messung], verglichen mit der Summe aus Kernel- und Transferzeit von Stufe 2. Die selbst gewählte Work-Group-Größe in Stufe 3 ändert die Leistung gegenüber Stufe 2 um [Messung] (RTX), [Messung] (L40S) und [Messung] (CPU). Die Kachelung in Stufe 4 steigert die Leistung gegenüber Stufe 2 um 31 % auf der RTX, auf der L40S sogar um 69 %. Auf der CPU erreicht Stufe 1 mit sechs Kernen rund 45 GFLOPS, etwa 90-mal so viel wie ein einzelner Kern. Die Kachelung steigert die Leistung dort auf mehr als das Dreifache, weil die Kacheln in die Caches der CPU passen. Der erste Lauf mit JIT-Kompilierung dauert bei $n = 1024$ zwei- bis achtmal so lang wie die folgenden.

	RTX 5060 Ti		**L40S**		**Ryzen 5 5600X**	
Stufe	$n = 1024$	$n = 2048$	$n = 1024$	$n = 2048$	$n = 1024$	$n = 2048$
0 (ein CPU-Kern)	0,5	–	0,3	–	0,5	–
1 (USM shared)	1105	1099	3459	2929	45	43
2 (USM device)	1088	1083	3402	2926	44	43
2b (Buffer, inkl. Transfer)	[?]	[?]	[?]	[?]	[?]	[?]
3 (nd_range, $W = 16$)	[?]	[?]	[?]	[?]	[?]	[?]
4 (Kachelung, $T = 16$)	1395	1418	4720	4934	154	150
5 (angepasst)	[?]	[?]	[?]	[?]	[?]	[?]

Tabelle 4: Leistung je Stufe in GFLOPS (Median aus fünf Läufen, Stufe 0 auf einem Kern der jeweiligen Host-CPU)

Tabelle 5 zeigt die zugehörigen Laufzeiten. Bei kleinen Matrizen kostet der Datentransfer zur GPU mehr als die Rechnung. Auf der RTX überwiegt die Rechenzeit ab $n = 1024$, auf der schnelleren L40S erst bei $n = 2048$. Auf der CPU ist der Transfer vernachlässigbar, da Host und Gerät denselben Arbeitsspeicher nutzen. Abbildung 9 zeigt die Effizienz je Kachelgröße auf allen drei Geräten. Die beste Kachelgröße ist geräteabhängig. Auf beiden GPUs ist $T = 16$ am schnellsten, auf der CPU $T = 32$. Bei $T = 64$ startet der Kernel auf den GPUs nicht, sodass die Performance-Portabilität (PP) auf null fällt. Nach Pennycook et al. [13] ist sie das harmonische Mittel der Effizienzen, die eine Anwendung auf den einzelnen Geräten einer Gerätemenge erreicht. Wird ein Gerät nicht unterstützt, ist sie null. Als Effizienz dient hier die Leistung relativ zur besten

Stufe	RTX 5060 Ti		L40S		Ryzen 5 5600X	
	$n = 1024$	$n = 2048$	$n = 1024$	$n = 2048$	$n = 1024$	$n = 2048$
0 (ein CPU-Kern)	4484	–	6650	–	4484	–
1 (USM shared)	1,94	15,64	0,62	5,87	47,6	401
2 Kernel	1,97	15,87	0,63	5,87	48,7	402
Transfer	1,34	3,93	0,60	2,12	0,90	5,68
2b Kernel und Transfer	[?]	[?]	[?]	[?]	[?]	[?]
3 Kernel ($W = 16$)	[?]	[?]	[?]	[?]	[?]	[?]
4 Kernel ($T = 16$)	1,54	12,12	0,46	3,48	14,0	114
Transfer	1,40	4,21	0,73	2,15	1,05	5,74
5 Kernel (angepasst)	[?]	[?]	[?]	[?]	[?]	[?]
erster Lauf mit Auswahl	[?]	[?]	[?]	[?]	[?]	[?]

Tabelle 5: Laufzeit je Stufe in ms (Median aus fünf Läufen). Ab Stufe 2 sind Kernel- und Transferzeit getrennt erfasst, außer bei Stufe 2b. Die Transferzeit von Stufe 3 und 5 entspricht der von Stufe 2 bzw. 4

Kachelgröße auf demselben Gerät. Den höchsten Wert von 0,99 erreicht $T = 32$.

Stufe 5 sortiert $T = 64$ auf beiden GPUs schon über die Geräteabfrage aus und startet deshalb auf allen drei Geräten. Gewählt wurde $T =$ [?] auf der RTX, $T =$ [?] auf der L40S und $T =$ [?] auf der CPU. Daraus ergibt sich eine Performance-Portabilität von [Messung] gegenüber 0,99 für das beste feste T und 0 für $T = 64$. Der Preis ist ein längerer erster Lauf, der bei $n = 1024$ wegen der Auswahl [Messung]-mal so lang dauert wie in Stufe 4 (Tabelle 5).

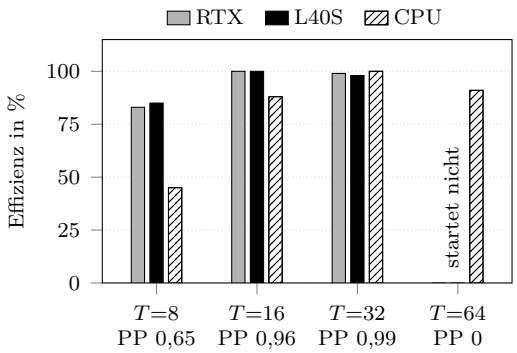


Abbildung 9: Effizienz und Performance-Portabilität je Kachelgröße bei $n = 2048$

8 Diskussion

Für F1 zeigen die Stufen, welches Konzept welches Problem löst (Tabelle 3). Herstellerbindung und mehrfache Codebasen löst bereits Stufe 1 durch Single-Source in Standard-C++, Backends und JIT-Kompilierung. Dasselbe Programm läuft ohne Neu-

übersetzung auf GPU und CPU. Den Zielkonflikt zwischen Abstraktion und Kontrolle entschärft SYCL durch einen gestuften Einstieg. Ein einfacher Indexraum mit USM shared genügt für ein lauffähiges Programm, während expliziter Transfer, Work-Groups und lokaler Speicher die Kontrolle für die Optimierung zurückgeben (Stufen 2 bis 4). Stufe 2b zeigt, dass SYCL den Konflikt dabei nicht nur in eine Richtung auflöst. Buffer und Accessoren nehmen dem Programmierer die Abhängigkeiten ab, kosten aber die Kontrolle über den Transfer, bis hin zu seiner getrennten Messbarkeit.

Für F2 bestätigt die Untersuchung funktionale Portabilität für alle Stufen. Performance-Portabilität ergibt sich dagegen nicht von selbst. SYCL abstrahiert die Schnittstelle, nicht die Hardware. Parameter wie die Work-Group-Größe bleiben gerätespezifisch, und ein für die CPU günstiger Wert kann auf der GPU sogar zum Abbruch führen. Dies deckt sich mit der in früheren Studien beobachteten Abhängigkeit von Implementierung und Parametrierung [11]. Selbst zwischen zwei NVIDIA-GPUs unterscheidet sich der Nutzen der Kachelung deutlich (31 % gegenüber 69 %), auch wenn die beste Kachelgröße übereinstimmt. Den größten Gewinn bringt bereits der erste, unveränderte Kernel. Die hardwarenahe Kachelung lohnt sich zusätzlich, bindet das Programm aber an gerätespezifische Parameter. Stufe 3 zeigt, wie viel davon schon auf die Work-Group-Größe allein entfällt. Stufe 5 zeigt, dass SYCL diese Bindung nicht auflöst, aber handhabbar macht. Geräteabfragen und Fehlerbehandlung verhindern den Abbruch, eine kurze Messung beim Start findet die passende Kachelgröße, und Specialization Constants erlauben das, ohne mehrere Varianten des Kernels im Quellcode zu halten. Dass AdaptiveCpp gerade diesen Mechanismus nicht unterstützt und DPC++ ihn für CUDA nur emuliert, zeigt aber, dass auch der Standard selbst nicht überall gleich weit umgesetzt ist. Die Kenntnis, welche Parameter überhaupt variiert werden müssen und welche Werte in Frage kommen, bleibt jedoch beim Programmierer.

Die Aussagekraft der Untersuchung ist begrenzt. Untersucht wurden ein einzelner Kernel, drei Geräte mit GPUs nur eines Herstellers und eine Implementierung. Die PP-Werte gelten nur für diese Gerätemenge [16], und ein Vergleich mit nativem CUDA-Code oder optimierten Bibliotheken fehlt. Die Zeiten wurden auf dem Host gemessen und enthalten damit das Einreichen der Aufträge. Eine Messung über die Profiling-Events von SYCL würde die reine Ausführungszeit auf dem Gerät liefern und vor allem die Werte für kleine Matrizen genauer machen.

9 Schlussfolgerung

SYCL erfüllt sein Versprechen „einmal schreiben, überall ausführen“ für die Korrektheit. Ein schrittweise portiertes Programm lief in allen Stufen unverändert auf CPU und GPU. Die gerätespezifische Abstimmung für hohe Leistung nimmt SYCL dem Programmierer jedoch nicht ab. Folgearbeiten sollten GPUs weiterer Hersteller, etwa

die AMD MI300A des DACHS-Clusters, und eine zweite Implementierung wie AdaptiveCpp einbeziehen.

Literaturverzeichnis

[1] J. Nickolls, I. Buck, M. Garland und K. Skadron, „Scalable Parallel Programming with CUDA," *ACM Queue*, Jg. 6, Nr. 2, S. 40–53, 2008. DOI: 10.1145/1365490.1365500.

[2] J. E. Stone, D. Gohara und G. Shi, „OpenCL: A Parallel Programming Standard for Heterogeneous Computing Systems," *Computing in Science & Engineering*, Jg. 12, Nr. 3, S. 66–73, 2010. DOI: 10.1109/MCSE.2010.69.

[3] The Khronos SYCL Working Group. „SYCL 2020 Specification," besucht am 30. Okt. 2026. Adresse: https://registry.khronos.org/SYCL/specs/sycl-2020/html/sycl-2020.html.

[4] Intel Corporation. „Intel LLVM-based Compiler (DPC++) – Repository," besucht am 30. Okt. 2026. Adresse: https://github.com/intel/llvm.

[5] A. Alpay und V. Heuveline, „SYCL beyond OpenCL: The Architecture, Current State and Future Direction of hipSYCL," in *Proceedings of the International Workshop on OpenCL (IWOCL '20)*, 2020. DOI: 10.1145/3388333.3388658.

[6] AdaptiveCpp Contributors. „AdaptiveCpp – Repository," besucht am 30. Okt. 2026. Adresse: https://github.com/AdaptiveCpp/AdaptiveCpp.

[7] A. Alpay und V. Heuveline, „One Pass to Bind Them: The First Single-Pass SYCL Compiler with Unified Code Representation Across Backends," in *Proceedings of the International Workshop on OpenCL (IWOCL '23)*, 2023.

[8] H. C. Edwards, C. R. Trott und D. Sunderland, „Kokkos: Enabling Manycore Performance Portability through Polymorphic Memory Access Patterns," *Journal of Parallel and Distributed Computing*, Jg. 74, Nr. 12, S. 3202–3216, 2014. DOI: 10.1016/j.jpdc.2014.07.003.

[9] D. A. Beckingsale u. a., „RAJA: Portable Performance for Large-Scale Scientific Applications," in *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*, 2019, S. 71–81.

[10] J. Reinders, B. Ashbaugh, J. Brodman, M. Kinsner, J. Pennycook und X. Tian, *Data Parallel C++: Programming Accelerated Systems Using C++ and SYCL*, 2. Aufl. Berkeley, CA: Apress, 2023, ISBN: 978-1-4842-9691-2. DOI: 10.1007/978-1-4842-9691-2.

[11] T. Deakin und S. McIntosh-Smith, „Evaluating the Performance of HPC-Style SYCL Applications," in *Proceedings of the International Workshop on OpenCL (IWOCL '20)*, 2020. DOI: 10.1145/3388333.3388643.

[12] M. Breyer, A. Van Craen und D. Pflüger, „A Comparison of SYCL, OpenCL, CUDA, and OpenMP for Massively Parallel Support Vector Machine Classification on Multi-Vendor Hardware," in *Proceedings of the International Workshop on OpenCL (IWOCL '22)*, 2022.

[13] S. J. Pennycook, J. D. Sewall und V. W. Lee, „A Metric for Performance Portability," in *Proceedings of the 7th International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2016. arXiv: 1611.07409.

[14] W.-m. W. Hwu, D. B. Kirk und I. El Hajj, *Programming Massively Parallel Processors: A Hands-on Approach*, 4. Aufl. Cambridge, MA: Morgan Kaufmann, 2022.

[15] NVIDIA Corporation. „CUDA C++ Programming Guide," besucht am 30. Okt. 2026. Adresse: `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`.

[16] S. J. Pennycook, J. D. Sewall und V. W. Lee, „Implications of a Metric for Performance Portability," *Future Generation Computer Systems*, Jg. 92, S. 947–958, 2019.

A Quellcode der Matrixmultiplikation

Die folgenden Listings zeigen den vollständigen Quellcode der Stufen 0 bis 5. Die Work-Group-Größe von Stufe 3 wird beim Übersetzen mit `-DWGSIZE=16` gesetzt, die Kachelgröße von Stufe 4 mit `-DTILE=8|16|32|64`. Stufe 5 wählt sie zur Laufzeit. Das Gerät wird für alle Stufen über `ONEAPI_DEVICE_SELECTOR` gewählt.

A.1 Stufe 0 – Sequentielles Ausgangsprogramm

```
// Stufe 0 - Das "existierende Programm"
//
// Klassische Matrixmultiplikation C = A * B in reinem C++, ein Kern,
// keine Parallelisierung. Dient als Ausgangspunkt und Referenz für
// Laufzeit und Korrektheit.
//
// Bild für die Folie: drei verschachtelte Schleifen. Jedes Element C[i][j]
// ist das Skalarprodukt aus Zeile i von A und Spalte j von B.

#include "common.hpp"

static void matmul(const float* A, const float* B, float* C, std::size_t n) {
  for (std::size_t i = 0; i < n; ++i)          // jede Zeile von C
    for (std::size_t j = 0; j < n; ++j) {      // jede Spalte von C
      float sum = 0.0f;
      for (std::size_t k = 0; k < n; ++k)      // Skalarprodukt
        sum += A[i * n + k] * B[k * n + j];
      C[i * n + j] = sum;
    }
}

int main(int argc, char** argv) {
  const auto cfg = bench::parse_args(argc, argv);
  const std::size_t n = cfg.n;

  std::vector<float> A(n * n), B(n * n), C(n * n);
  bench::fill_random(A.data(), n * n, 1);
  bench::fill_random(B.data(), n * n, 2);

  // Erster Lauf (zum Vergleich mit dem "ersten Lauf" der SYCL-Stufen)
  bench::Timer t0;
  matmul(A.data(), B.data(), C.data(), n);
  const double first = t0.ms();

  std::vector<double> times;
  for (int r = 0; r < cfg.reps; ++r) {
    bench::Timer t;
    matmul(A.data(), B.data(), C.data(), n);
    times.push_back(t.ms());
  }

  const bool ok = bench::verify(A.data(), B.data(), C.data(), n);
  bench::report("0_sequentiell", "CPU (1 Kern)", n, first, bench::median(times), 0.0, ok);
  return ok ? 0 : 1;
}
```

A.2 Stufe 1 – USM shared und `parallel_for`

```
// Stufe 1 - Der erste SYCL-Kernel
//
// Kleinstmögliche Änderung gegenüber Stufe 0:
//   * eine Queue anlegen (sie wählt das Gerät, z. B. die GPU)
//   * Speicher mit malloc_shared anlegen (USM "shared"): derselbe Zeiger
//     ist auf Host und Gerät gültig, die Laufzeit verschiebt die Daten
//     automatisch dorthin, wo sie gebraucht werden
//   * die beiden äußeren Schleifen (i, j) durch parallel_for ersetzen -
//     jedes C[i][j] wird von einem eigenen Work-Item berechnet
//
// Die innere Schleife (k) bleibt unverändert im Kernel.
//
// Neue Konzepte: queue, device, kernel, parallel_for, range/id, USM shared

#include <sycl/sycl.hpp>
#include "common.hpp"

int main(int argc, char** argv) {
  const auto cfg = bench::parse_args(argc, argv);
  const std::size_t n = cfg.n;

  // 1) Queue: Verbindung zu einem Gerät. Ohne Angabe nimmt SYCL das
  //    "beste" verfügbare Gerät (meist die GPU).
  sycl::queue q;
  const std::string dev = q.get_device().get_info<sycl::info::device::name>();

  // 2) Gemeinsamer Speicher (Unified Shared Memory, Variante "shared")
  float* A = sycl::malloc_shared<float>(n * n, q);
  float* B = sycl::malloc_shared<float>(n * n, q);
  float* C = sycl::malloc_shared<float>(n * n, q);
  bench::fill_random(A, n * n, 1);
  bench::fill_random(B, n * n, 2);

  // 3) Kernel: ersetzt die i- und j-Schleife aus Stufe 0
  auto run = [&] {
    q.parallel_for(sycl::range<2>{n, n}, [=](sycl::id<2> idx) {
       const std::size_t i = idx[0];
       const std::size_t j = idx[1];
       float sum = 0.0f;
       for (std::size_t k = 0; k < n; ++k)
         sum += A[i * n + k] * B[k * n + j];
       C[i * n + j] = sum;
     }).wait();  // auf das Ende des Kernels warten
  };

  // Erster Lauf: enthält ggf. JIT-Kompilierung des Kernels und das erste
  // Verschieben der Daten auf das Gerät -> separat ausweisen!
  bench::Timer t0;
  run();
  const double first = t0.ms();

  std::vector<double> times;
  for (int r = 0; r < cfg.reps; ++r) {
    bench::Timer t;
    run();
    times.push_back(t.ms());
  }
```

```
58
59    // Zugriff auf C vom Host aus: mit malloc_shared einfach möglich
60    const bool ok = bench::verify(A, B, C, n);
61    bench::report("1_usm_shared", dev, n, first, bench::median(times), 0.0, ok);
62
63    sycl::free(A, q);
64    sycl::free(B, q);
65    sycl::free(C, q);
66    return ok ? 0 : 1;
67  }
```

A.3 Stufe 2 – USM device und explizites Kopieren

```
1   // Stufe 2 - Speicher selbst verwalten
2   //
3   // Unterschied zu Stufe 1: Statt malloc_shared (automatisches Verschieben)
4   // wird der Speicher explizit getrennt:
5   //   * Host-Speicher (normales std::vector) für Initialisierung und Prüfung
6   //   * Geräte-Speicher (malloc_device), nur auf dem Gerät gültig
7   //   * Kopieren mit q.memcpy, Host -> Gerät und Gerät -> Host
8   //
9   // Der Kernel selbst ist identisch zu Stufe 1.
10  //
11  // Ziel für die Präsentation: Datentransfer wird sichtbar und messbar.
12  // Gerade bei kleinen Matrizen kostet das Kopieren oft mehr als die Rechnung.
13  //
14  // Neue Konzepte: malloc_device, memcpy, Trennung Host-/Geräte-Speicher
15
16  #include <sycl/sycl.hpp>
17  #include "common.hpp"
18
19  int main(int argc, char** argv) {
20    const auto cfg = bench::parse_args(argc, argv);
21    const std::size_t n = cfg.n;
22    const std::size_t bytes = n * n * sizeof(float);
23
24    sycl::queue q;
25    const std::string dev = q.get_device().get_info<sycl::info::device::name>();
26
27    // Host-Seite
28    std::vector<float> hA(n * n), hB(n * n), hC(n * n);
29    bench::fill_random(hA.data(), n * n, 1);
30    bench::fill_random(hB.data(), n * n, 2);
31
32    // Geräte-Seite
33    float* A = sycl::malloc_device<float>(n * n, q);
34    float* B = sycl::malloc_device<float>(n * n, q);
35    float* C = sycl::malloc_device<float>(n * n, q);
36
37    auto copy_in = [&] {
38      q.memcpy(A, hA.data(), bytes);
39      q.memcpy(B, hB.data(), bytes);
40      q.wait();
41    };
42    auto kernel = [&] {
43      q.parallel_for(sycl::range<2>{n, n}, [=](sycl::id<2> idx) {
44         const std::size_t i = idx[0];
```

```
45          const std::size_t j = idx[1];
46          float sum = 0.0f;
47          for (std::size_t k = 0; k < n; ++k)
48            sum += A[i * n + k] * B[k * n + j];
49          C[i * n + j] = sum;
50        }).wait();
51    };
52    auto copy_out = [&] { q.memcpy(hC.data(), C, bytes).wait(); };
53
54    // Erster Lauf komplett (Kopieren + Kernel inkl. JIT + Zurückkopieren)
55    bench::Timer t0;
56    copy_in();
57    kernel();
58    copy_out();
59    const double first = t0.ms();
60
61    // Messläufe: Kernel und Transfer getrennt erfassen
62    std::vector<double> kernel_times, transfer_times;
63    for (int r = 0; r < cfg.reps; ++r) {
64      bench::Timer tin;
65      copy_in();
66      const double in_ms = tin.ms();
67
68      bench::Timer tk;
69      kernel();
70      kernel_times.push_back(tk.ms());
71
72      bench::Timer tout;
73      copy_out();
74      transfer_times.push_back(in_ms + tout.ms());
75    }
76
77    const bool ok = bench::verify(hA.data(), hB.data(), hC.data(), n);
78    bench::report("2_usm_device", dev, n, first, bench::median(kernel_times),
79                  bench::median(transfer_times), ok);
80
81    sycl::free(A, q);
82    sycl::free(B, q);
83    sycl::free(C, q);
84    return ok ? 0 : 1;
85  }
```

A.4 Stufe 2b – Buffer und Accessoren

```
1   // Stufe 2b - Variante von Stufe 2: Buffer und Accessoren statt USM
2   //
3   // Gleicher Kernel wie in Stufe 1 und 2, aber die Daten werden nicht über
4   // Zeiger, sondern über Buffer verwaltet:
5   //   * Ein Buffer kapselt die Daten eines Host-Felds
6   //   * Der Kernel greift nur über Accessoren zu, die angeben, ob gelesen oder
7   //     geschrieben wird (read_only, write_only)
8   //   * Aus diesen Angaben leitet die Laufzeit selbst ab, wann kopiert werden
9   //     muss und welche Aufträge voneinander abhängen. Es gibt weder memcpy
10  //     noch ein explizites wait.
11  //   * Wird der Buffer zerstört, schreibt die Laufzeit das Ergebnis in das
12  //     Host-Feld zurück und wartet darauf.
13  //
```

```cpp
// Preis der Abstraktion: Kopieren und Rechnen lassen sich nicht mehr
// getrennt messen. median_ms enthält hier Transfer UND Kernel.
//
// Neue Konzepte: buffer, accessor, Zugriffsmodi, implizite Abhängigkeiten

#include <sycl/sycl.hpp>
#include "common.hpp"

int main(int argc, char** argv) {
  const auto cfg = bench::parse_args(argc, argv);
  const std::size_t n = cfg.n;

  sycl::queue q;
  const std::string dev = q.get_device().get_info<sycl::info::device::name>();

  std::vector<float> hA(n * n), hB(n * n), hC(n * n);
  bench::fill_random(hA.data(), n * n, 1);
  bench::fill_random(hB.data(), n * n, 2);

  // Ein vollständiger Durchlauf: Buffer anlegen, Kernel einreichen, beim
  // Verlassen des Blocks schreiben die Buffer-Destruktoren C zurück.
  auto lauf = [&] {
    sycl::buffer<float, 1> bA(hA.data(), sycl::range<1>{n * n});
    sycl::buffer<float, 1> bB(hB.data(), sycl::range<1>{n * n});
    sycl::buffer<float, 1> bC(hC.data(), sycl::range<1>{n * n});

    q.submit([&](sycl::handler& h) {
      sycl::accessor A(bA, h, sycl::read_only);
      sycl::accessor B(bB, h, sycl::read_only);
      sycl::accessor C(bC, h, sycl::write_only, sycl::no_init);

      h.parallel_for(sycl::range<2>{n, n}, [=](sycl::id<2> idx) {
        const std::size_t i = idx[0];
        const std::size_t j = idx[1];
        float sum = 0.0f;
        for (std::size_t k = 0; k < n; ++k)
          sum += A[i * n + k] * B[k * n + j];
        C[i * n + j] = sum;
      });
    });
  };  // <- hier: Kopie von C zum Host, Host wartet

  bench::Timer t0;
  lauf();
  const double first = t0.ms();

  // Messläufe: nur die Gesamtzeit (Transfer + Kernel) ist messbar
  std::vector<double> times;
  for (int r = 0; r < cfg.reps; ++r) {
    bench::Timer t;
    lauf();
    times.push_back(t.ms());
  }

  const bool ok = bench::verify(hA.data(), hB.data(), hC.data(), n);
  bench::report("2b_buffer", dev, n, first, bench::median(times), 0.0, ok);
  return ok ? 0 : 1;
}
```

A.5 Stufe 3 – Work-Groups mit `nd_range`

```
// Stufe 3 - Work-Groups festlegen (nd_range), noch ohne lokalen Speicher
//
// Unterschied zu Stufe 2: Der Indexraum wird nicht mehr als einfache range,
// sondern als nd_range angegeben. Damit legt das Programm selbst fest, wie
// viele Work-Items eine Work-Group bilden (hier W x W). In Stufe 1 und 2
// hat das die Laufzeit entschieden.
//
// Der Rechenteil ist identisch mit Stufe 2: Jedes Work-Item liest weiterhin
// eine ganze Zeile von A und Spalte von B aus dem globalen Speicher. So
// lässt sich getrennt messen, was die Work-Group-Größe allein bringt, bevor
// Stufe 4 den lokalen Speicher hinzunimmt.
//
// Speicherverwaltung wie Stufe 2.
//
// Neue Konzepte: nd_range, nd_item, Work-Group, globale/lokale ID
//
// Work-Group-Größe beim Kompilieren ändern, z. B. -DWGSIZE=8 oder -DWGSIZE=32.

#include <sycl/sycl.hpp>
#include "common.hpp"

#ifndef WGSIZE
#define WGSIZE 16
#endif

int main(int argc, char** argv) {
  const auto cfg = bench::parse_args(argc, argv);
  const std::size_t n = cfg.n;
  const std::size_t bytes = n * n * sizeof(float);
  constexpr std::size_t W = WGSIZE;

  sycl::queue q;
  const std::string dev = q.get_device().get_info<sycl::info::device::name>();

  std::vector<float> hA(n * n), hB(n * n), hC(n * n);
  bench::fill_random(hA.data(), n * n, 1);
  bench::fill_random(hB.data(), n * n, 2);

  float* A = sycl::malloc_device<float>(n * n, q);
  float* B = sycl::malloc_device<float>(n * n, q);
  float* C = sycl::malloc_device<float>(n * n, q);

  // Der globale Bereich muss ein Vielfaches der Work-Group-Größe sein
  const std::size_t np = (n + W - 1) / W * W;

  auto copy_in = [&] {
    q.memcpy(A, hA.data(), bytes);
    q.memcpy(B, hB.data(), bytes);
    q.wait();
  };
  auto kernel = [&] {
    q.parallel_for(
         sycl::nd_range<2>{{np, np}, {W, W}},  // global, lokal (Work-Group)
         [=](sycl::nd_item<2> it) {
           const std::size_t i = it.get_global_id(0);
           const std::size_t j = it.get_global_id(1);
           if (i >= n || j >= n) return;   // Work-Items außerhalb der Matrix
```

```
            float sum = 0.0f;
            for (std::size_t k = 0; k < n; ++k)
              sum += A[i * n + k] * B[k * n + j];
            C[i * n + j] = sum;
          })
         .wait();
  };
  auto copy_out = [&] { q.memcpy(hC.data(), C, bytes).wait(); };

  bench::Timer t0;
  copy_in();
  kernel();
  copy_out();
  const double first = t0.ms();

  std::vector<double> kernel_times, transfer_times;
  for (int r = 0; r < cfg.reps; ++r) {
    bench::Timer tin;
    copy_in();
    const double in_ms = tin.ms();

    bench::Timer tk;
    kernel();
    kernel_times.push_back(tk.ms());

    bench::Timer tout;
    copy_out();
    transfer_times.push_back(in_ms + tout.ms());
  }

  const bool ok = bench::verify(hA.data(), hB.data(), hC.data(), n);
  bench::report("3_ndrange_WG" + std::to_string(W), dev, n, first,
                bench::median(kernel_times), bench::median(transfer_times), ok);

  sycl::free(A, q);
  sycl::free(B, q);
  sycl::free(C, q);
  return ok ? 0 : 1;
}
```

A.6 Stufe 4 – Kachelung im lokalen Speicher

```
// Stufe 4 - Hardwarenah optimieren: Local Memory (Tiling)
//
// Problem der Stufen 1 bis 3: Jedes Work-Item liest seine komplette Zeile
// von A und Spalte von B aus dem langsamen globalen Speicher. Benachbarte
// Work-Items lesen dabei immer wieder dieselben Werte.
//
// Idee: Die Matrix wird in Kacheln (Tiles) der Größe TILE x TILE zerlegt.
//   * Work-Items werden zu Work-Groups von TILE x TILE zusammengefasst
//   * Jede Work-Group lädt gemeinsam eine Kachel von A und eine von B in
//     den schnellen Local Memory (auf der GPU: Shared Memory des Rechenblocks)
//   * Barriere: warten, bis alle Werte geladen sind
//   * Jedes Work-Item rechnet mit den Kacheldaten weiter
//   * Barriere, dann die nächste Kachel laden
// Dadurch wird jeder Wert aus dem globalen Speicher nur noch einmal pro
// Work-Group statt einmal pro Work-Item gelesen.
```

```
16 //
17 // Speicher wie Stufe 2, nd_range wie Stufe 3 - neu ist nur der Kernel.
18 //
19 // Neue Konzepte: Command Group (submit mit handler), local_accessor (Local
20 // Memory), group_barrier
21 //
22 // Kachelgröße beim Kompilieren ändern, z. B. -DTILE=8 oder -DTILE=32.
23 // Die beste Wahl hängt vom Gerät ab (-> Performance-Portabilität!).
24
25 #include <sycl/sycl.hpp>
26 #include "common.hpp"
27
28 #ifndef TILE
29 #define TILE 16
30 #endif
31
32 int main(int argc, char** argv) {
33   const auto cfg = bench::parse_args(argc, argv);
34   const std::size_t n = cfg.n;
35   const std::size_t bytes = n * n * sizeof(float);
36   constexpr std::size_t T = TILE;
37
38   sycl::queue q;
39   const std::string dev = q.get_device().get_info<sycl::info::device::name>();
40
41   std::vector<float> hA(n * n), hB(n * n), hC(n * n);
42   bench::fill_random(hA.data(), n * n, 1);
43   bench::fill_random(hB.data(), n * n, 2);
44
45   float* A = sycl::malloc_device<float>(n * n, q);
46   float* B = sycl::malloc_device<float>(n * n, q);
47   float* C = sycl::malloc_device<float>(n * n, q);
48
49   // Globaler Bereich auf ein Vielfaches der Kachelgröße aufrunden, damit
50   // auch n, die nicht durch TILE teilbar sind, funktionieren.
51   const std::size_t np = (n + T - 1) / T * T;
52
53   auto copy_in = [&] {
54     q.memcpy(A, hA.data(), bytes);
55     q.memcpy(B, hB.data(), bytes);
56     q.wait();
57   };
58
59   auto kernel = [&] {
60     q.submit([&](sycl::handler& h) {
61       // Local Memory: eine Kachel von A und eine von B pro Work-Group
62       sycl::local_accessor<float, 2> tileA({T, T}, h);
63       sycl::local_accessor<float, 2> tileB({T, T}, h);
64
65       h.parallel_for(
66           sycl::nd_range<2>{{np, np}, {T, T}},  // global, lokal (Work-Group)
67           [=](sycl::nd_item<2> it) {
68             const std::size_t i = it.get_global_id(0);   // Zeile in C
69             const std::size_t j = it.get_global_id(1);   // Spalte in C
70             const std::size_t li = it.get_local_id(0);   // Position in
71             const std::size_t lj = it.get_local_id(1);   // der Kachel
72
73             float sum = 0.0f;
74             for (std::size_t t = 0; t < n; t += T) {
```

```
75                // gemeinsam eine Kachel laden (außerhalb der Matrix: 0)
76                tileA[li][lj] = (i < n && t + lj < n) ? A[i * n + t + lj] : 0.0f;
77                tileB[li][lj] = (t + li < n && j < n) ? B[(t + li) * n + j] : 0.0f;
78                sycl::group_barrier(it.get_group());
79
80                // mit den Daten aus dem schnellen Local Memory rechnen
81                for (std::size_t k = 0; k < T; ++k)
82                  sum += tileA[li][k] * tileB[k][lj];
83                sycl::group_barrier(it.get_group());
84              }
85              if (i < n && j < n) C[i * n + j] = sum;
86            });
87      }).wait();
88    };
89
90    auto copy_out = [&] { q.memcpy(hC.data(), C, bytes).wait(); };
91
92    bench::Timer t0;
93    copy_in();
94    kernel();
95    copy_out();
96    const double first = t0.ms();
97
98    std::vector<double> kernel_times, transfer_times;
99    for (int r = 0; r < cfg.reps; ++r) {
100     bench::Timer tin;
101     copy_in();
102     const double in_ms = tin.ms();
103
104     bench::Timer tk;
105     kernel();
106     kernel_times.push_back(tk.ms());
107
108     bench::Timer tout;
109     copy_out();
110     transfer_times.push_back(in_ms + tout.ms());
111   }
112
113   const bool ok = bench::verify(hA.data(), hB.data(), hC.data(), n);
114   bench::report("4_tiling_T" + std::to_string(T), dev, n, first,
115                 bench::median(kernel_times), bench::median(transfer_times), ok);
116
117   sycl::free(A, q);
118   sycl::free(B, q);
119   sycl::free(C, q);
120   return ok ? 0 : 1;
121 }
```

A.7 Stufe 5 – Anpassung an das Gerät

```
1  // Stufe 5 - An das Gerät anpassen: Geräteabfragen, Specialization Constants,
2  // Fehlerbehandlung
3  //
4  // Problem der Stufe 4: Die Kachelgröße T ist beim Übersetzen fest. Die beste
5  // Wahl hängt aber vom Gerät ab, und ein zu großes T (z. B. 64 -> 4096
6  // Work-Items je Work-Group) startet auf der GPU gar nicht, das Programm
7  // stürzt ab.
```

```
8  //
9  // Idee: T wird erst zur Laufzeit festgelegt, und zwar
10 //   1. über eine Specialization Constant: Der Host setzt T vor dem Start,
11 //      der JIT-Compiler setzt den Wert dann als Konstante in den Kernel ein.
12 //      Ein Quellcode, eine ausführbare Datei, trotzdem ein konstantes T.
13 //   2. nach einer Geräteabfrage: Kachelgrößen, die das Gerät nicht erlaubt,
14 //      werden gar nicht erst versucht.
15 //   3. nach einer kurzen Messung der übrigen Kandidaten (je drei Läufe).
16 // Fehler werden abgefangen (synchron per try/catch, asynchron über einen
17 // Async Handler), statt das Programm zu beenden.
18 //
19 // Kernel und Speicherverwaltung sonst wie in Stufe 4.
20 //
21 // Neue Konzepte: Geräteabfragen, specialization_id, kernel_handler,
22 // sycl::exception, Async Handler, wait_and_throw
23
24 #include <sycl/sycl.hpp>
25 #include <exception>
26 #include "common.hpp"
27
28 #ifndef __ACPP__
29 // Specialization Constant für die Kachelgröße (SYCL 2020, Vorgabe 16)
30 constexpr sycl::specialization_id<std::size_t> tile_id{16};
31 #endif
32
33 // Kandidaten für die Kachelgröße
34 constexpr std::size_t kandidaten[] = {8, 16, 32, 64};
35
36 // Kernel aus Stufe 4. Die Kachelgröße kommt als Argument für den Host
37 // (Größe von nd_range und lokalem Speicher) und als Specialization Constant
38 // für den Kernel.
39 sycl::event matmul_tiled(sycl::queue& q, const float* A, const float* B, float* C,
40                          std::size_t n, std::size_t tile) {
41   const std::size_t np = (n + tile - 1) / tile * tile;
42 #ifdef __ACPP__
43   // AdaptiveCpp kennt specialization_id nicht, bietet aber eine eigene
44   // Erweiterung mit derselben Wirkung
45   const sycl::specialized<std::size_t> tile_spez{tile};
46 #endif
47   return q.submit([&](sycl::handler& h) {
48 #ifndef __ACPP__
49     h.set_specialization_constant<tile_id>(tile);
50 #endif
51     sycl::local_accessor<float, 2> tileA({tile, tile}, h);
52     sycl::local_accessor<float, 2> tileB({tile, tile}, h);
53
54     h.parallel_for(
55         sycl::nd_range<2>{{np, np}, {tile, tile}},
56 #ifdef __ACPP__
57         [=](sycl::nd_item<2> it) {
58           const std::size_t T = tile_spez;
59 #else
60         [=](sycl::nd_item<2> it, sycl::kernel_handler kh) {
61           // Für den JIT-Compiler eine Konstante
62           const std::size_t T = kh.get_specialization_constant<tile_id>();
63 #endif
64           const std::size_t i = it.get_global_id(0);
65           const std::size_t j = it.get_global_id(1);
66           const std::size_t li = it.get_local_id(0);
```

```
67            const std::size_t lj = it.get_local_id(1);
68
69            float sum = 0.0f;
70            for (std::size_t t = 0; t < n; t += T) {
71              tileA[li][lj] = (i < n && t + lj < n) ? A[i * n + t + lj] : 0.0f;
72              tileB[li][lj] = (t + li < n && j < n) ? B[(t + li) * n + j] : 0.0f;
73              sycl::group_barrier(it.get_group());
74
75              for (std::size_t k = 0; k < T; ++k)
76                sum += tileA[li][k] * tileB[k][lj];
77              sycl::group_barrier(it.get_group());
78            }
79            if (i < n && j < n) C[i * n + j] = sum;
80          });
81    });
82  }
83
84  int main(int argc, char** argv) {
85    const auto cfg = bench::parse_args(argc, argv);
86    const std::size_t n = cfg.n;
87    const std::size_t bytes = n * n * sizeof(float);
88
89    // Async Handler: Fehler, die erst während der Ausführung auf dem Gerät
90    // auftreten, sammelt die Laufzeit und übergibt sie hier, sobald der Host
91    // mit wait_and_throw() wartet. Wir geben sie als Ausnahme weiter, damit sie
92    // genauso behandelt werden wie synchrone Fehler.
93    auto async_handler = [](sycl::exception_list fehler) {
94      for (const std::exception_ptr& e : fehler) std::rethrow_exception(e);
95    };
96
97    try {
98      sycl::queue q{sycl::default_selector_v, async_handler};
99      const sycl::device d = q.get_device();
100     const std::string dev = d.get_info<sycl::info::device::name>();
101
102     // 1) Geräteabfragen: Grenzen des gewählten Geräts
103     const std::size_t max_wg = d.get_info<sycl::info::device::max_work_group_size>();
104     const sycl::range<2> max_items =
105         d.get_info<sycl::info::device::max_work_item_sizes<2>>();
106     const std::size_t local_mem = d.get_info<sycl::info::device::local_mem_size>();
107     std::fprintf(stderr,
108                  "[stufe5] %s: max. Work-Group %zu, max. je Dimension %zu x %zu, "
109                  "lokaler Speicher %zu Byte\n",
110                  dev.c_str(), max_wg, max_items[0], max_items[1], local_mem);
111
112     std::vector<float> hA(n * n), hB(n * n), hC(n * n);
113     bench::fill_random(hA.data(), n * n, 1);
114     bench::fill_random(hB.data(), n * n, 2);
115
116     float* A = sycl::malloc_device<float>(n * n, q);
117     float* B = sycl::malloc_device<float>(n * n, q);
118     float* C = sycl::malloc_device<float>(n * n, q);
119
120     auto copy_in = [&] {
121       q.memcpy(A, hA.data(), bytes);
122       q.memcpy(B, hB.data(), bytes);
123       q.wait_and_throw();
124     };
125     auto copy_out = [&] {
```

```
126       q.memcpy(hC.data(), C, bytes);
127       q.wait_and_throw();
128     };
129
130     bench::Timer t0;
131     copy_in();
132
133     // 2) Auswahl: erst Kachelgrößen aussortieren, die das Gerät laut Abfrage
134     //    nicht erlaubt, dann die übrigen kurz messen. Der erste Aufruf je
135     //    Kachelgröße enthält die JIT-Übersetzung und wird nicht gewertet.
136     std::size_t T = 0;
137     double beste_ms = 0.0;
138     for (const std::size_t t : kandidaten) {
139       const bool passt = t * t <= max_wg && t <= max_items[0] && t <= max_items[1] &&
140                          2 * t * t * sizeof(float) <= local_mem;
141       if (!passt) {
142         std::fprintf(stderr, "[stufe5] T = %zu: vom Gerät nicht erlaubt\n", t);
143         continue;
144       }
145       try {
146         matmul_tiled(q, A, B, C, n, t);
147         q.wait_and_throw();
148         double ms = 1e300;                    // schnellster von drei Läufen
149         for (int r = 0; r < 3; ++r) {
150           bench::Timer tv; matmul_tiled(q, A, B, C, n, t); q.wait_and_throw();
151           ms = std::min(ms, tv.ms()); }
152         std::fprintf(stderr, "[stufe5] T = %zu: %.3f ms\n", t, ms);
153         if (T == 0 || ms < beste_ms) {
154           T = t;
155           beste_ms = ms;
156         }
157       } catch (const sycl::exception& e) {
158         // Die Geräteabfrage liefert nur Obergrenzen. Scheitert der Kernel
159         // trotzdem, wird die nächste Kachelgröße versucht.
160         std::fprintf(stderr, "[stufe5] T = %zu: startet nicht (%s)\n", t, e.what());
161       }
162     }
163     if (T == 0) {
164       std::fprintf(stderr, "[stufe5] keine Kachelgröße läuft auf %s\n", dev.c_str());
165       sycl::free(A, q);
166       sycl::free(B, q);
167       sycl::free(C, q);
168       return 2;
169     }
170     std::fprintf(stderr, "[stufe5] gewählt: T = %zu (Auswahl %.1f ms)\n", T, t0.ms());
171
172     auto kernel = [&] {
173       matmul_tiled(q, A, B, C, n, T);
174       q.wait_and_throw();
175     };
176
177     // Erster Lauf: hier einschließlich Abfrage und Auswahl
178     kernel();
179     copy_out();
180     const double first = t0.ms();
181
182     std::vector<double> kernel_times, transfer_times;
183     for (int r = 0; r < cfg.reps; ++r) {
184       bench::Timer tin;
```

```
185       copy_in();
186       const double in_ms = tin.ms();
187
188       bench::Timer tk;
189       kernel();
190       kernel_times.push_back(tk.ms());
191
192       bench::Timer tout;
193       copy_out();
194       transfer_times.push_back(in_ms + tout.ms());
195     }
196
197     const bool ok = bench::verify(hA.data(), hB.data(), hC.data(), n);
198     bench::report("5_angepasst_T" + std::to_string(T), dev, n, first,
199                   bench::median(kernel_times), bench::median(transfer_times), ok);
200
201     sycl::free(A, q);
202     sycl::free(B, q);
203     sycl::free(C, q);
204     return ok ? 0 : 1;
205   } catch (const sycl::exception& e) {
206     // 3) Alles, was außerhalb der Auswahl schiefgeht (kein Gerät, Speicher
207     //    voll, Fehler im gewählten Kernel), beendet das Programm geordnet.
208     std::fprintf(stderr, "[stufe5] SYCL-Fehler: %s\n", e.what());
209     return 2;
210   }
211 }
```

A.8 Gemeinsame Hilfsfunktionen (`common.hpp`)

```
1  // common.hpp - gemeinsame Hilfsfunktionen für alle Stufen
2  //
3  // Enthält: Kommandozeile lesen, Matrizen befüllen, Zeitmessung,
4  // Ergebnisprüfung und eine einheitliche CSV-Ausgabe, damit die
5  // Ergebnisse aller Stufen direkt vergleichbar sind.
6  #pragma once
7
8  #include <algorithm>
9  #include <chrono>
10 #include <cmath>
11 #include <cstdio>
12 #include <cstdlib>
13 #include <random>
14 #include <string>
15 #include <vector>
16
17 namespace bench {
18
19 // Einstellungen aus der Kommandozeile:  ./stufeX <n> [wiederholungen]
20 struct Config {
21   std::size_t n = 1024;   // Matrixgröße n x n
22   int reps = 5;           // Messwiederholungen (nach dem Aufwärmlauf)
23 };
24
25 inline Config parse_args(int argc, char** argv) {
26   Config c;
27   if (argc > 1) c.n = std::strtoul(argv[1], nullptr, 10);
```

```
28    if (argc > 2) c.reps = std::atoi(argv[2]);
29    if (c.n == 0 || c.reps < 1) {
30      std::fprintf(stderr, "Aufruf: %s <n> [wiederholungen]\n", argv[0]);
31      std::exit(1);
32    }
33    return c;
34  }
35
36  // Matrizen mit reproduzierbaren Zufallswerten im Bereich [-1, 1] füllen.
37  inline void fill_random(float* m, std::size_t count, unsigned seed) {
38    std::mt19937 gen(seed);
39    std::uniform_real_distribution<float> dist(-1.0f, 1.0f);
40    for (std::size_t i = 0; i < count; ++i) m[i] = dist(gen);
41  }
42
43  // Einfache Stoppuhr in Millisekunden.
44  class Timer {
45   public:
46    Timer() : start_(clock::now()) {}
47    double ms() const {
48      return std::chrono::duration<double, std::milli>(clock::now() - start_).count();
49    }
50   private:
51    using clock = std::chrono::steady_clock;
52    clock::time_point start_;
53  };
54
55  inline double median(std::vector<double> v) {
56    std::sort(v.begin(), v.end());
57    const std::size_t m = v.size() / 2;
58    return v.size() % 2 ? v[m] : 0.5 * (v[m - 1] + v[m]);
59  }
60
61  // Ergebnisprüfung: 64 zufällige Einträge von C werden in double-Genauigkeit
62  // nachgerechnet. Das kostet nur O(64 * n) und funktioniert auch für große n.
63  inline bool verify(const float* A, const float* B, const float* C, std::size_t n) {
64    std::mt19937 gen(42);
65    std::uniform_int_distribution<std::size_t> idx(0, n - 1);
66    for (int s = 0; s < 64; ++s) {
67      const std::size_t i = idx(gen), j = idx(gen);
68      double ref = 0.0;
69      for (std::size_t k = 0; k < n; ++k) ref += double(A[i * n + k]) * double(B[k * n + j]);
70      const double got = C[i * n + j];
71      const double tol = 1e-3 * std::max(1.0, std::sqrt(double(n)));
72      if (std::fabs(got - ref) > tol) {
73        std::fprintf(stderr, "FEHLER bei C[%zu][%zu]: %f statt %f\n", i, j, got, ref);
74        return false;
75      }
76    }
77    return true;
78  }
79
80  // Einheitliche Ergebniszeile (CSV), damit sich alle Stufen in eine Tabelle
81  // schreiben lassen:
82  //   stufe;geraet;n;erster_lauf_ms;median_ms;transfer_ms;gflops;korrekt
83  inline void report(const std::string& stage, const std::string& device, std::size_t n,
84                     double first_ms, double median_ms, double transfer_ms, bool ok) {
85    const double gflops = 2.0 * double(n) * double(n) * double(n) / (median_ms * 1e6);
86    std::printf("%s;%s;%zu;%.3f;%.3f;%.3f;%.2f;%s\n", stage.c_str(), device.c_str(), n,
```

```
87                first_ms, median_ms, transfer_ms, gflops, ok ? "ja" : "NEIN");
88  }
89
90  }  // namespace bench
```